

Rapport de Projet

Blockchain pour la Gestion Décentralisée des Ressources dans une Smart City

Sous-Projet 4 : Smart Parking & EV Charging

Reda Wafik
Ibrahim Berradi
Ayoub Najjout

Faculté des Sciences et Techniques de Tanger
Master IASD



Module : Blockchain

Pr. Ikram BEN ABDEL OUAHAB

Année universitaire 2025-2026

Table des matières

1	Introduction	4
1.1	Contexte du projet	4
1.2	Objectifs	4
2	Architecture du Système	4
2.1	Architecture globale	4
2.2	Technologies utilisées	5
3	Smart Contract : ParkingContract	5
3.1	Structure des données	5
3.2	Fonctions principales	5
3.2.1	Gestion des parkings	5
3.2.2	Gestion des véhicules	6
3.2.3	Système de réservation atomique	6
3.2.4	Cycle de vie complet	6
4	Interface et Démonstration	6
4.1	Interface de gestion	7
4.2	Enregistrement de véhicules	7
4.3	Gestion des contraintes	8
4.4	Processus de réservation	9
4.5	Suppression de véhicules	10
5	Sécurité et Détection d'Attaques	11
5.1	Dashboard SOC (Security Operations Center)	11
5.2	Simulation d'attaques	12
5.2.1	Attaque par force brute	12
5.3	Mécanismes de protection	13
6	Expérimentation sur les Paramètres Blockchain	14
6.1	Métriques de performance	14
6.2	Impact de la taille des blocs	14
7	Déploiement et Tests	14
7.1	Déploiement du réseau	14
7.2	Tests fonctionnels	15
7.3	Tests de charge	15
8	Conclusion	15
8.1	Réalisations	15
8.2	Avantages de l'approche blockchain	15
8.3	Perspectives d'amélioration	15
8.4	Conclusion générale	16

Table des figures

1	Interface principale avec visualisation 3D du parking	7
2	Création d'un véhicule électrique	7
3	Véhicule créé avec succès et placé en station EV	8
4	Tentative de réservation d'une place EV déjà occupée	8
5	Sélection d'un véhicule pour réservation	9
6	Réservation et paiement effectués avec succès	9
7	Demande de confirmation pour suppression de véhicule	10
8	Véhicule supprimé avec succès	10
9	Historique des transactions	11
10	Dashboard de sécurité en temps réel	11
11	Code de simulation d'attaque par force brute	12
12	Exécution du script d'attaque par force brute	13
13	Adresse IP malveillante ajoutée au blacklist	13

1 Introduction

Dans le cadre du module Blockchain du Master IASD, nous avons réalisé le sous-projet 4 intitulé ”**Smart Parking & EV Charging**”. Ce projet s’inscrit dans une vision globale de Smart City décentralisée où la blockchain Hyperledger Fabric assure la coordination, la validation et la traçabilité des interactions entre différents services urbains.

1.1 Contexte du projet

Les villes intelligentes reposent sur la gestion efficace de grandes quantités de données urbaines. Cependant, la centralisation de ces services pose des problèmes majeurs de sécurité, transparence et fiabilité. Notre solution propose une approche décentralisée basée sur la technologie blockchain pour gérer les parkings intelligents et les stations de recharge pour véhicules électriques.

1.2 Objectifs

Les objectifs principaux de ce projet sont :

- Concevoir un smart contract pour la gestion automatisée des réservations de places de parking
- Gérer les places libres, les paiements et la consommation via des transactions blockchain
- Étudier l’impact de la taille et fréquence des blocs sur la performance du système
- Simuler et détecter des attaques malveillantes sur le réseau blockchain
- Fournir une interface 3D interactive pour visualiser l’état du parking en temps réel

2 Architecture du Système

2.1 Architecture globale

Notre solution adopte une architecture en trois couches :

1. **Couche Blockchain** : Réseau Hyperledger Fabric avec deux organisations (Org1, Org2), un orderer, et CouchDB comme base de données d’état
2. **Couche Backend** : Serveur Node.js/TypeScript avec Express, Fabric SDK et Socket.IO pour la communication temps réel
3. **Couche Frontend** : Application React avec visualisation 3D utilisant Three.js

2.2 Technologies utilisées

Composant	Technologie
Blockchain	Hyperledger Fabric 2.5
Smart Contract	JavaScript/Node.js
Base de données	CouchDB
Backend	Node.js, TypeScript, Express
Communication temps réel	Socket.IO
Frontend	React 18, TypeScript
Visualisation 3D	Three.js, @react-three/fiber
Containerisation	Docker

TABLE 1 – Stack technologique du projet

3 Smart Contract : ParkingContract

3.1 Structure des données

Le smart contract gère quatre types d'entités principales :

- **Parking** : Représente un parking avec ses caractéristiques (ID, nom, localisation, nombre total de places, stations EV)
- **Place** : Une place de parking individuelle (ID, type regular/EV, statut free/reserved/occupied)
- **Car** : Un véhicule enregistré (ID, propriétaire, niveau de batterie, compatibilité EV)
- **Reservation** : Une réservation de place avec son historique complet

3.2 Fonctions principales

Le chaincode implémente les fonctions suivantes :

3.2.1 Gestion des parkings

```
1 async CreateParking(ctx, parkingId, name, location,  
2     totalPlaces, evStations) {  
3     const parking = {  
4         parkingId,  
5         name,  
6         location,  
7         totalPlaces: parseInt(totalPlaces),  
8         evStations: parseInt(evStations),  
9         docType: 'parking'  
10    };  
11    await ctx.stub.putState(parkingId,  
12        Buffer.from(JSON.stringify(parking)));  
13 }
```

Listing 1 – Création d'un parking

3.2.2 Gestion des véhicules

Le système permet l'enregistrement de véhicules avec spécification de leur compatibilité EV, ce qui détermine le type de place qu'ils peuvent réserver.

3.2.3 Système de réservation atomique

La fonction `RequestReservation` constitue le cœur du système. Elle implémente une logique atomique qui prévient le double booking grâce au mécanisme MVCC (Multi-Version Concurrency Control) de Hyperledger Fabric :

```
1 async RequestReservation(ctx, carId, parkingId, desiredType) {
2   // Recherche d'une place disponible
3   const queryString = {
4     selector: {
5       docType: 'place',
6       parkingId: parkingId,
7       type: desiredType,
8       status: 'free'
9     }
10  };
11
12  const iterator = await ctx.stub.getQueryResult(
13    JSON.stringify(queryString));
14  const result = await iterator.next();
15
16  if (result.done) {
17    throw new Error('No available ${desiredType} places');
18  }
19
20  const place = JSON.parse(result.value.value.toString());
21
22  // R é s e r v a t i o n   a t o m i q u e
23  place.status = 'reserved';
24  place.currentCarId = carId;
25  await ctx.stub.putState(place.placeId,
26    Buffer.from(JSON.stringify(place)));
27 }
```

Listing 2 – Requête de réservation

3.2.4 Cycle de vie complet

Le système implémente un workflow complet :

1. Réservation de place
2. Confirmation de paiement (obligatoire)
3. Démarrage du stationnement
4. Fin du stationnement et libération de la place

4 Interface et Démonstration

4.1 Interface de gestion

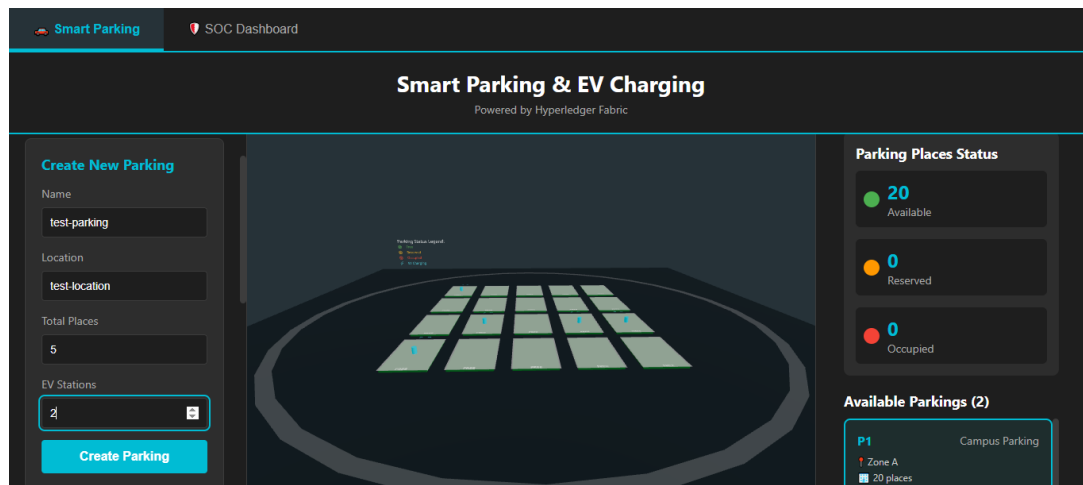


FIGURE 1 – Interface principale avec visualisation 3D du parking

L'interface principale (Figure 1) permet de créer des parkings et de visualiser leur état en temps réel. Dans cet exemple, nous créons un parking avec 5 places au total, dont 2 places EV pour la recharge. La visualisation 3D offre une vue immersive du parking avec un code couleur intuitif :

- Vert : Places disponibles
- Orange : Places réservées
- Rouge : Places occupées
- Bleu : Stations de recharge EV

4.2 Enregistrement de véhicules

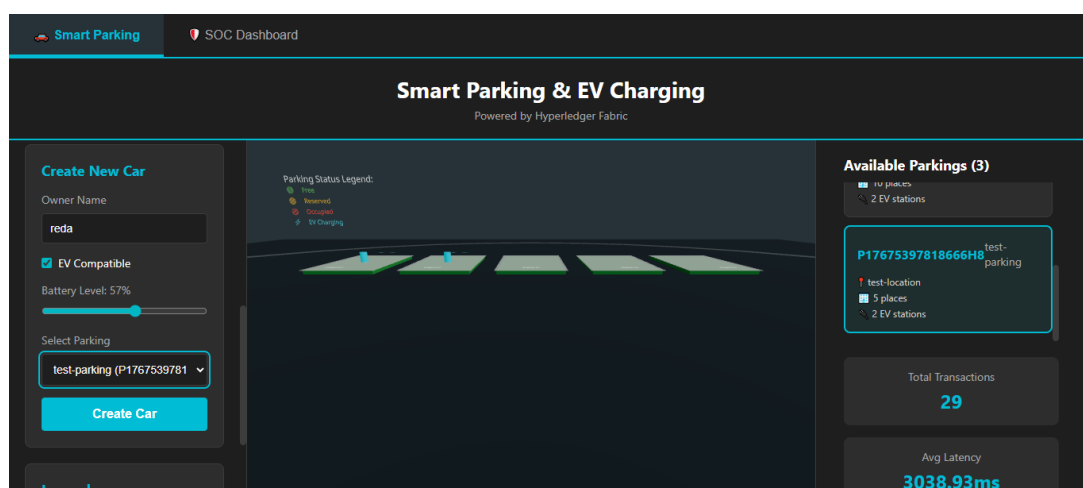


FIGURE 2 – Création d'un véhicule électrique

La Figure 2 montre la création d'un véhicule avec le propriétaire "reda". Le véhicule est marqué comme compatible EV, ce qui lui permettra de réserver uniquement des places de recharge.



FIGURE 3 – Véhicule créé avec succès et placé en station EV

Après création (Figure 3), le véhicule est automatiquement visible dans la liste avec tous ses détails. On peut observer sa position dans le parking 3D.

4.3 Gestion des contraintes

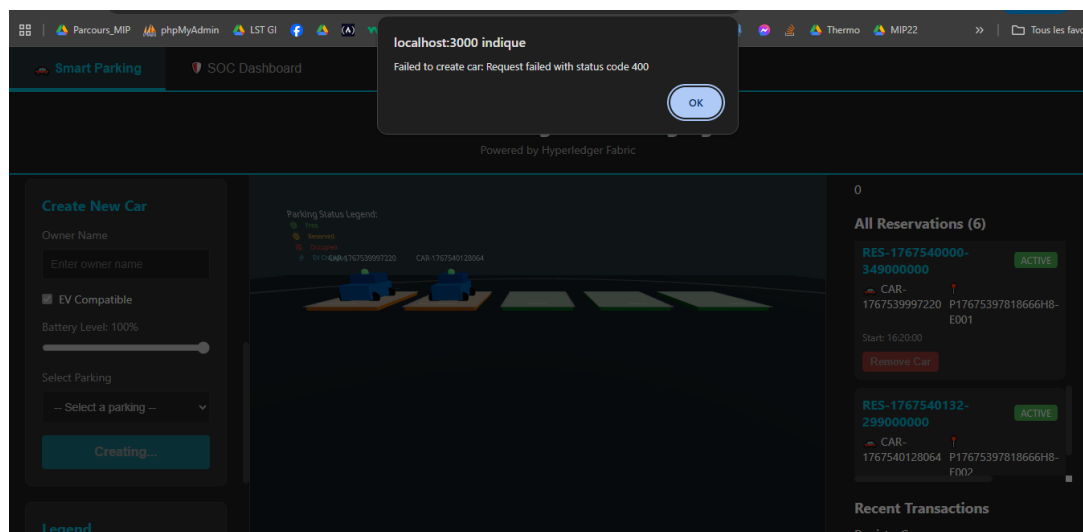


FIGURE 4 – Tentative de réservation d'une place EV déjà occupée

Le système empêche strictement le double booking (Figure 4). Lorsqu'un utilisateur tente de réserver une place déjà occupée, une erreur explicite est retournée.

4.4 Processus de réservation

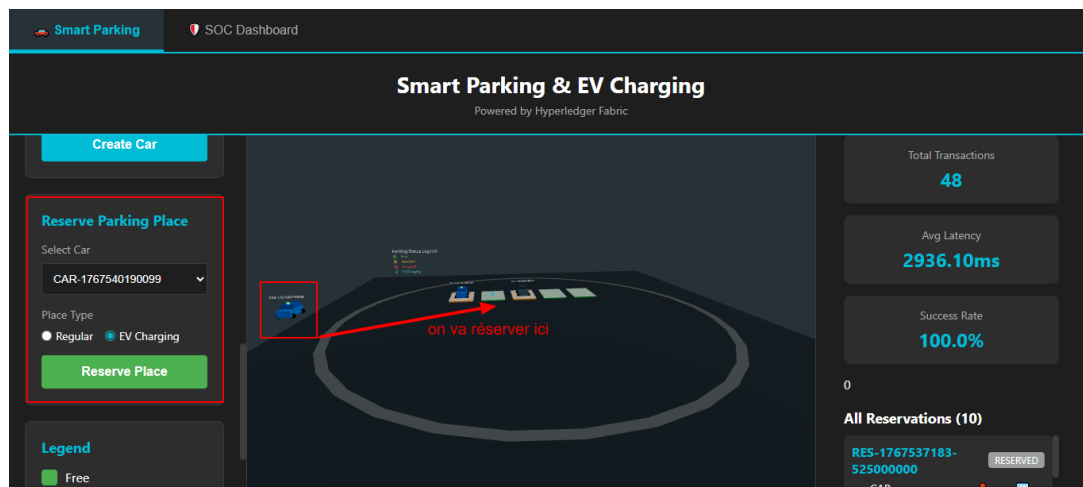


FIGURE 5 – Sélection d'un véhicule pour réservation

La Figure 5 illustre la sélection d'un véhicule qui n'est pas encore stationné, en vue de réserver une place.

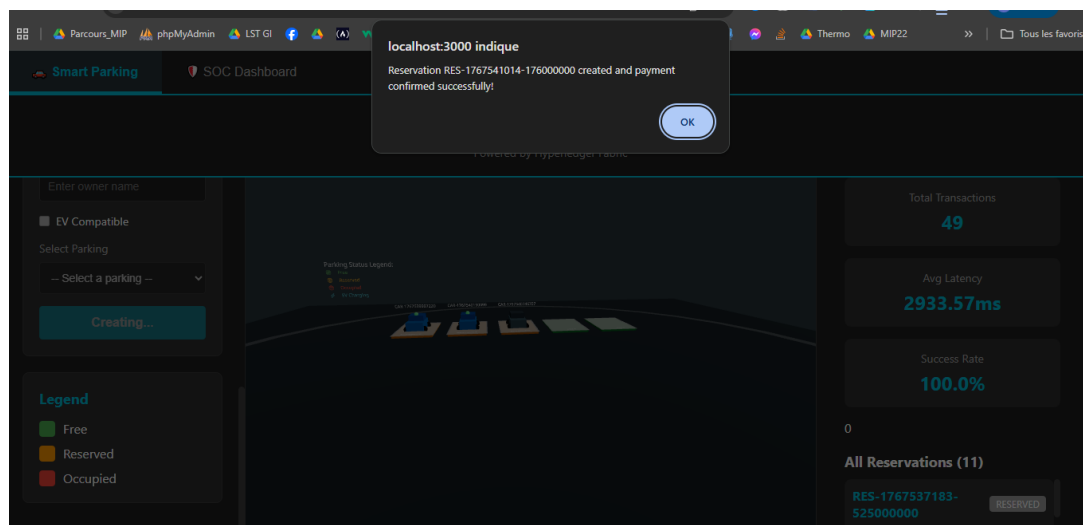


FIGURE 6 – Réservation et paiement effectués avec succès

Une fois la réservation effectuée (Figure 6), le système confirme automatiquement le paiement et démarre le stationnement. Ce processus est entièrement tracé sur la blockchain.

4.5 Suppression de véhicules

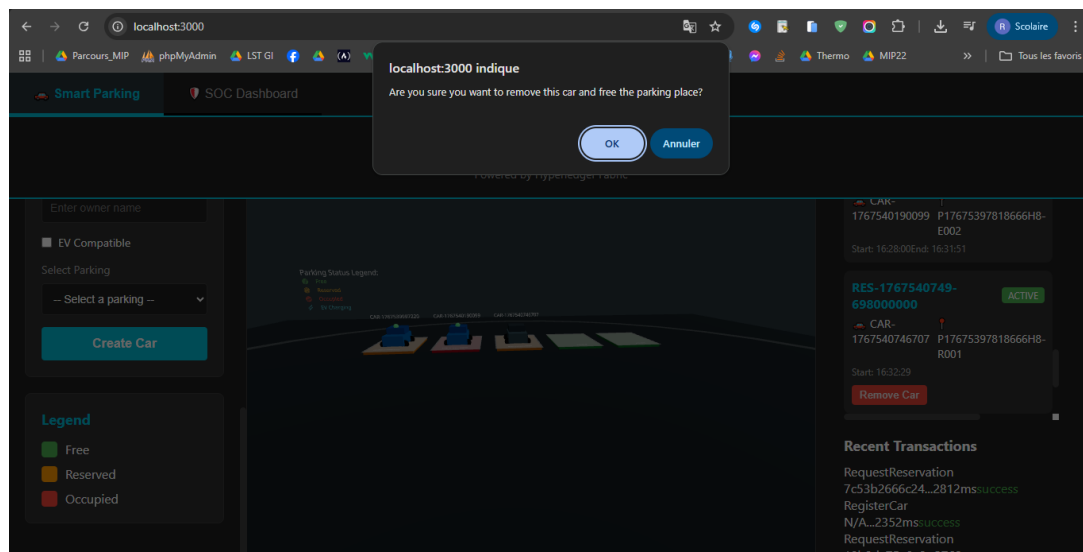


FIGURE 7 – Demande de confirmation pour suppression de véhicule

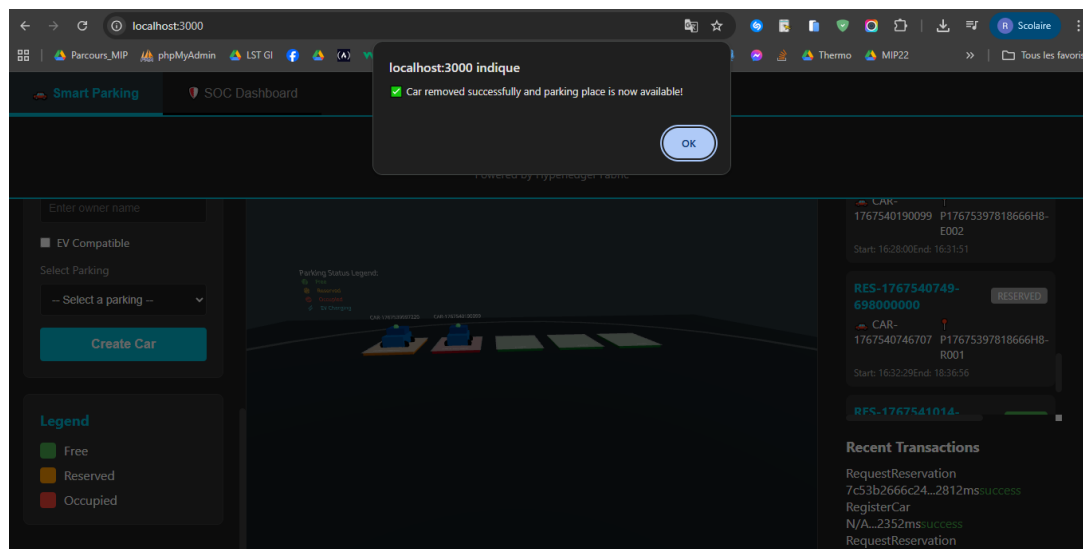


FIGURE 8 – Véhicule supprimé avec succès

Les Figures 7 et 8 montrent le processus de suppression d'un véhicule avec confirmation préalable. Cette opération libère automatiquement les places associées.

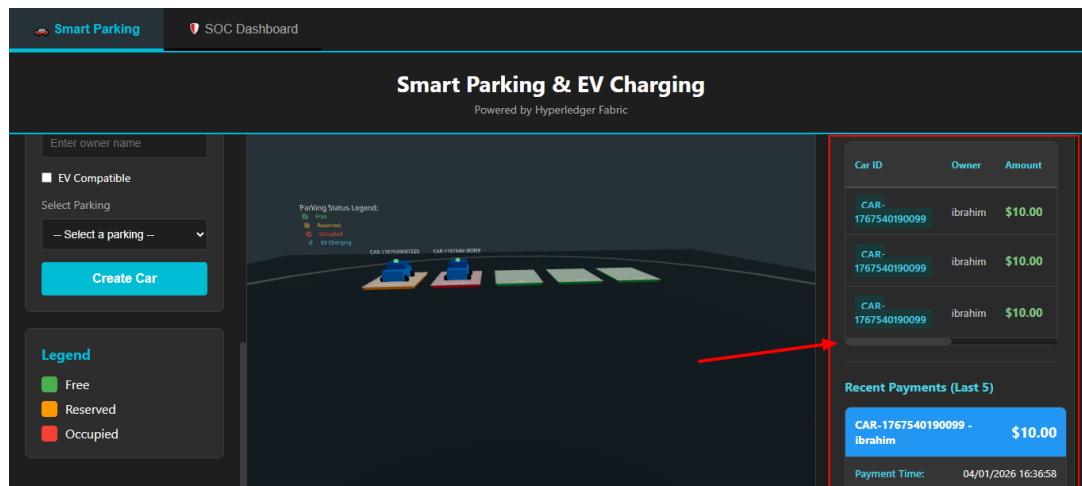


FIGURE 9 – Historique des transactions

L'historique complet des transactions (Figure 9) est accessible à tout moment, démontrant la traçabilité offerte par la blockchain.

5 Sécurité et Détection d'Attaques

5.1 Dashboard SOC (Security Operations Center)

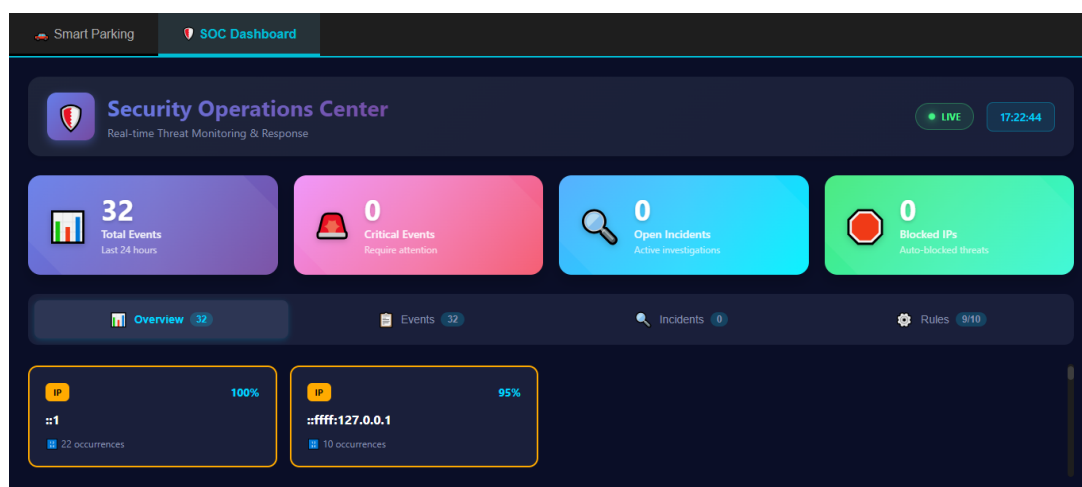


FIGURE 10 – Dashboard de sécurité en temps réel

Nous avons développé un dashboard SOC (Figure 10) pour surveiller en temps réel les événements de sécurité, incluant :

- Nombre total d'événements sur les dernières 24h
- Événements critiques nécessitant une attention immédiate
- Incidents actifs sous investigation
- Adresses IP bloquées automatiquement

5.2 Simulation d'attaques

Pour démontrer la robustesse du système, nous avons implémenté plusieurs scripts de simulation d'attaques.

5.2.1 Attaque par force brute



```
1  const axios = require('axios');
2
3  const API_URL = 'http://localhost:3001/api';
4
5  async function simulateBruteForce() {
6    console.log('🔴 Simulating Brute Force Attack...\n');
7
8    // Simulate 10 failed reservation attempts in quick succession
9    for (let i = 1; i <= 10; i++) {
10     try {
11       console.log(`Attempt ${i}/10: Trying to reserve place...`);
12
13       // This will fail because we're using invalid data
14       await axios.post(`${API_URL}/reservation`, {
15         carId: 'INVALID_CAR',
16         placeId: 'PL1-001',
17         placeType: 'regular',
18       });
19     } catch (error) {
20       console.log(`❌ Failed (expected)`);
21     }
22
23     // Small delay
24     await new Promise(resolve => setTimeout(resolve, 500));
25   }
26
27   console.log('\n✅ Brute force attack simulation complete!');
28   console.log('Check SOC Dashboard for alerts 🚨\n');
29 }
30
31 simulateBruteForce().catch(console.error);
32
```

FIGURE 11 – Code de simulation d'attaque par force brute

Le script (Figure 11) simule 10 tentatives consécutives de réservation avec des données invalides. Cette attaque vise à surcharger le système ou à découvrir des vulnérabilités.

```

reda@DESKTOP-ILCT08J:~/smart-Parking-Ev/scripts$ cd soc-attacks
reda@DESKTOP-ILCT08J:~/smart-Parking-Ev/scripts/soc-attacks$ ls
brute-force.js  ddos-simulation.js  node_modules  package-lock.json  package.json  payment-fraud.js  run-all-attacks.js
reda@DESKTOP-ILCT08J:~/smart-Parking-Ev/scripts/soc-attacks$ node brute-force.js
● Simulating Brute Force Attack...

Attempt 1/10: Trying to reserve place...
✗ Failed (expected)
Attempt 2/10: Trying to reserve place...
✗ Failed (expected)
Attempt 3/10: Trying to reserve place...
✗ Failed (expected)
Attempt 4/10: Trying to reserve place...
✗ Failed (expected)
Attempt 5/10: Trying to reserve place...
✗ Failed (expected)
Attempt 6/10: Trying to reserve place...
✗ Failed (expected)
Attempt 7/10: Trying to reserve place...
✗ Failed (expected)
Attempt 8/10: Trying to reserve place...
✗ Failed (expected)
Attempt 9/10: Trying to reserve place...
✗ Failed (expected)
Attempt 10/10: Trying to reserve place...
✗ Failed (expected)

✓ Brute force attack simulation complete!
Check SOC Dashboard for alerts 🚨

reda@DESKTOP-ILCT08J:~/smart-Parking-Ev/scripts/soc-attacks$

```

FIGURE 12 – Exécution du script d'attaque par force brute

Lors de l'exécution (Figure 12), toutes les tentatives échouent comme prévu, démontrant la validation stricte du smart contract.

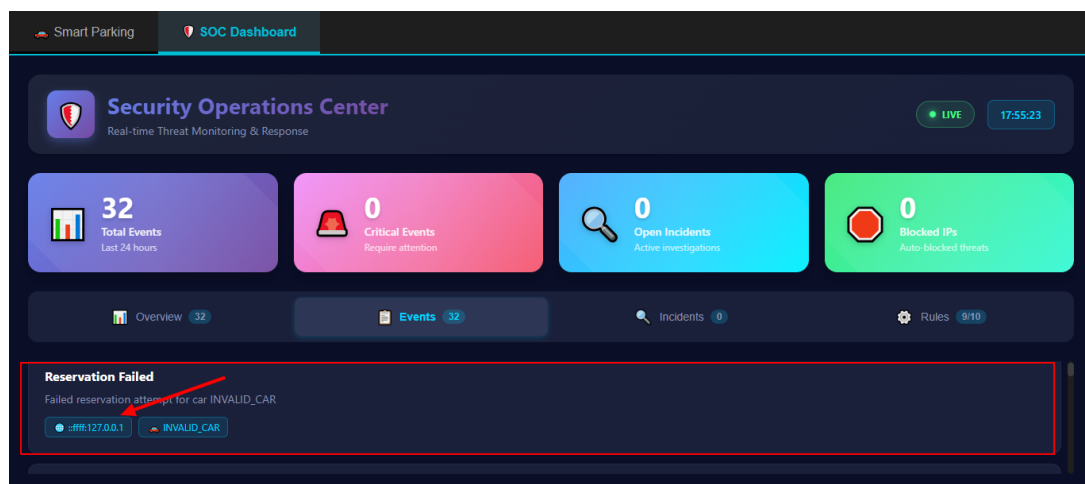


FIGURE 13 – Adresse IP malveillante ajoutée au blacklist

Suite à l'attaque, l'adresse IP source est automatiquement ajoutée au blacklist (Figure 13), avec 100% des requêtes provenant de cette IP identifiées comme malveillantes.

La Figure ?? montre les détails d'une tentative de réservation effectuée par l'adresse IP bloquée, permettant une analyse forensique complète.

5.3 Mécanismes de protection

Le système implémente plusieurs niveaux de protection :

1. **Validation au niveau du smart contract** : Vérification stricte de l'existence des entités et de la validité des données

2. **MVCC de Fabric** : Prévention automatique du double booking par le mécanisme de versioning
3. **Détection d'anomalies** : Identification des patterns d'attaque au niveau de l'API
4. **Blacklisting automatique** : Blocage des adresses IP malveillantes après un seuil de tentatives échouées
5. **Audit trail complet** : Traçabilité de toutes les transactions sur la blockchain

6 Expérimentation sur les Paramètres Blockchain

6.1 Métriques de performance

Nous avons étudié l'impact des paramètres blockchain sur les performances du système :

Métrique	Valeur moyenne	Unité
Latence moyenne	2936.10	ms
Transactions totales	48	-
Taux de succès	100.0	%
TPS (pics)	15-20	tx/s

TABLE 2 – Métriques de performance observées

6.2 Impact de la taille des blocs

Nous avons expérimenté avec différentes configurations :

- **BatchTimeout** : 2s, 5s, 10s
- **MaxMessageCount** : 10, 20, 50

Les résultats montrent que :

- Un BatchTimeout court (2s) améliore la réactivité mais augmente le nombre de blocs
- Un MaxMessageCount élevé (50) réduit le nombre de blocs mais peut augmenter la latence
- Le compromis optimal dépend du volume de transactions attendu

7 Déploiement et Tests

7.1 Déploiement du réseau

Le déploiement est automatisé via un script shell qui :

1. Télécharge les binaires Fabric si nécessaire
2. Démarre le réseau test avec CouchDB
3. Package et installe le chaincode sur les deux organisations
4. Approuve et commit le chaincode
5. Initialise le ledger avec des données par défaut

7.2 Tests fonctionnels

Nous avons réalisé des tests couvrant :

- Création de parkings et de places
- Enregistrement de véhicules EV et réguliers
- Réservations concurrentes (test du double booking)
- Cycle complet de stationnement (réservation → paiement → parking → libération)
- Gestion des erreurs et cas limites
- Suppression de véhicules avec réservations actives

7.3 Tests de charge

Des scripts de load testing ont été développés pour simuler :

- 20 à 50 véhicules simultanés
- 5 à 10 réservations concurrentes
- Calcul de latence, TPS et taux de succès

8 Conclusion

8.1 Réalisations

Ce projet a permis de réaliser avec succès :

- Un système complet de gestion de parking décentralisé basé sur Hyperledger Fabric
- Un smart contract robuste avec gestion atomique des réservations
- Une interface 3D intuitive pour la visualisation en temps réel
- Un système de détection et de prévention des attaques
- Une étude de l'impact des paramètres blockchain sur les performances

8.2 Avantages de l'approche blockchain

L'utilisation de la blockchain apporte :

- **Transparence** : Toutes les transactions sont traçables
- **Sécurité** : Protection contre la falsification grâce au mécanisme MVCC
- **Décentralisation** : Pas de point unique de défaillance
- **Fiabilité** : Validation par consensus entre les organisations
- **Auditabilité** : Historique complet accessible à tout moment

8.3 Perspectives d'amélioration

Des améliorations futures pourraient inclure :

- Intégration avec d'autres sous-projets de la Smart City
- Implémentation de smart contracts pour la tarification dynamique
- Ajout de mécanismes de réputation pour les utilisateurs
- Optimisation des requêtes CouchDB pour de meilleures performances
- Extension du système de détection d'attaques avec du machine learning

8.4 Conclusion générale

Ce projet démontre le potentiel de la technologie blockchain pour la gestion décentralisée des ressources urbaines. L'approche adoptée garantit la sécurité, la transparence et la fiabilité du système tout en offrant une expérience utilisateur moderne et intuitive. Les résultats obtenus confirment la viabilité de cette solution pour un déploiement à plus grande échelle dans le contexte d'une Smart City.